

Manual técnico

C++ → WebAssembly

→ GitHub Pages

Compilación, formato binario, publicación y ejecución en el navegador

Caso de estudio: Hands-on 1 (análisis léxico, AFD y máquina de Moore)

Alondra Goretti Gonzalez Martinez · Emiliano Barraza Cervantes

Profesor: José Antonio Aviña Méndez

Versión 1.0 · Octubre 2026

Contenido

1. Propósito, alcance y visión general	3
2. Fundamentos: qué es WebAssembly y por qué funciona	4
3. Anatomía del binario real (hands_on1.wasm)	6
4. Preparar el compilador (Emscripten SDK)	8
5. Preparar el código C++ para el navegador	9
6. Compilar paso a paso	11
7. Cómo el glue carga y arranca el binario	13
8. index.html: conectar la interfaz con el motor	14
9. Probar localmente antes de publicar	16
10. Git y GitHub: publicar los archivos	18
11. GitHub Pages: cómo se sirve el binario	20
12. Qué ocurre cuando alguien abre la URL	22
13. Cómo se construyó la práctica (procedimiento)	23
14. Solución de problemas	24
Apéndice A. Resumen de comandos	25
Apéndice B. Glosario	25

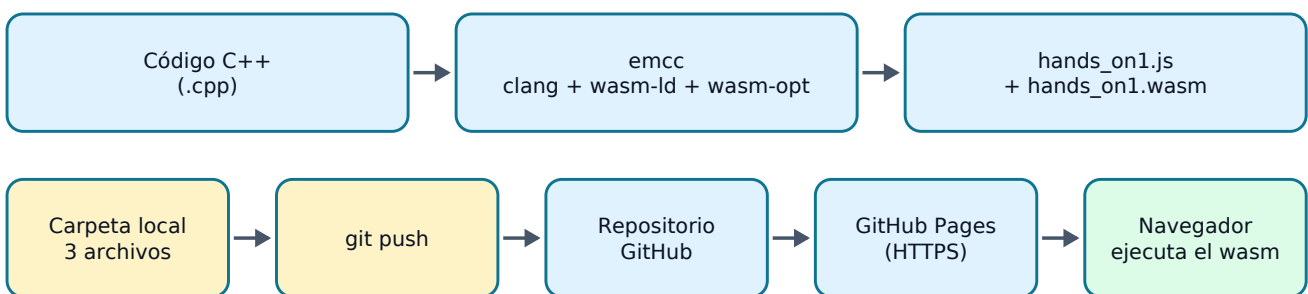
Consejo: Todo lo que este manual dice sobre *tus* archivos (tamaños, secciones del binario, funciones exportadas, salidas de prueba) se midió sobre el archivo entregado **hands_on1.zip**. Los comandos genéricos (instalar Emscripten, Git, GitHub Pages) son los procedimientos estándar; el compilador no se volvió a ejecutar para generar este documento, así que compara los números que obtengas con los tuyos.

1. Propósito, alcance y visión general

Este manual explica, paso a paso y con detalle técnico, cómo el motor en C++ de la práctica Hands-on 1 se convierte en un **módulo binario WebAssembly** que cualquier navegador puede ejecutar, cómo ese binario se **publica mediante GitHub** y cómo **GitHub Pages** lo entrega para que se ejecute en la computadora del visitante. No vuelve a explicar el autómata ni la máquina de Moore (ver el README del proyecto); explica el *proceso y la maquinaria alrededor de ellos*.

1.1 La idea en un párrafo

El código fuente C++ se compila **una sola vez**, en la máquina del desarrollador, con el compilador Emscripten (**emcc**). El resultado son dos archivos: un binario **hands_on1.wasm** (el programa) y un archivo de texto **hands_on1.js** (el «pegamento» que lo carga). Junto con **index.html** son archivos estáticos comunes. Se suben a un repositorio de GitHub y GitHub Pages los sirve por HTTPS. Cuando una persona abre la página, **su navegador** descarga los tres archivos, valida y compila el binario a código máquina nativo y lo ejecuta. **Ningún servidor ejecuta C++**: GitHub solo entrega archivos.



1.2 Inventario de la práctica

Archivo	Tamaño (bytes)	Función	¿Se publica?
HandsON1_gonzalez_martinez.cpp	10,369	Código fuente de todo el motor (AFD, objeto instrucción, Moore, salidas, interfaz wasm).	Recomendado
hands_on1.wasm	145,029	El binario compilado. Lo ejecuta el navegador.	Obligatorio
hands_on1.js	22,694	Pegamento de Emscripten: carga e instancia el .wasm, expone las exportaciones.	Obligatorio
index.html	13,258	Interfaz de usuario; habla con el wasm a través del glue.	Obligatorio
README.md	10,756	Registro técnico de la solución.	Recomendado
main.wasm	265,885	Una compilación distinta/anterior. No la referencia index.html ni hands_on1.js.	No

2. Fundamentos: qué es WebAssembly y por qué funciona

2.1 El problema: los navegadores no pueden ejecutar tu ejecutable de g++

Cuando compilas con `g++` obtienes código máquina para una familia de procesadores (x86-64, ARM...) y un sistema operativo (Windows PE, Linux ELF...). Una página web debe funcionar en cualquier dispositivo, por eso los navegadores solo ejecutan **JavaScript** y **WebAssembly (wasm)**. WebAssembly es un **formato binario de instrucciones portable** para una máquina virtual pequeña y segura. No depende de ningún procesador: cada navegador lo traduce a las instrucciones nativas de la CPU del visitante.

2.2 Propiedades clave

- **Binario y compacto:** un módulo empieza con los 4 bytes mágicos `00 61 73 6D` (el texto «\0asm») y un número de versión; está diseñado para decodificarse y compilarse muy rápido.
- **Tipado y validado:** solo existen cuatro tipos de valor (`i32`, `i64`, `f32`, `f64`). El navegador *valida* el módulo completo antes de ejecutar una sola instrucción; un archivo inválido se rechaza.
- **Memoria lineal aislada:** el programa ve un único arreglo contiguo de bytes (su «memoria lineal»). Los apuntadores de C++ son solo desplazamientos enteros en él. El módulo no puede leer la memoria de la página ni el disco del usuario.
- **Sin entrada/salida propia:** `wasm` no puede tocar el DOM, imprimir, leer archivos ni llamar al sistema operativo. Todo lo externo llega por *importaciones* que provee el anfitrión (JavaScript).
- **Velocidad casi nativa:** los motores compilan `wasm` a código máquina antes de ejecutarlo, por lo que es mucho más rápido que interpretar.

2.3 Quién hace qué

Pieza	Responsabilidad en esta práctica
Código C++ (compilado)	Toda la lógica: AFD, tokens, objeto instrucción, Moore, salida JSON.
<code>hands_on1.wasm</code>	La lógica compilada + las bibliotecas de ejecución de C/C++ que necesita (<code>libc</code> , <code>libc++</code>).
<code>hands_on1.js</code> (glue)	Descarga el <code>.wasm</code> , arma las importaciones, crea/enlaza la memoria, ejecuta constructores estáticos, expone <code>Module._h1_*</code> .
<code>index.html</code>	Interfaz. Envía texto al <code>wasm</code> , recibe JSON, dibuja el grafo, tokens, objeto, Moore y registros.
Navegador	Valida, compila y ejecuta el <code>wasm</code> en un entorno aislado; ejecuta JavaScript.

2.4 Dentro del compilador: qué hace realmente emcc

Emscripten es una cadena de herramientas, no un solo programa. `emcc` es un controlador que se comporta como `gcc` pero coordina varias herramientas:

1. **Front-end clang:** analiza C++17, produce código intermedio LLVM y luego un archivo objeto WebAssembly (destino `wasm32`).
2. **wasm-ld (enlazador):** une tu objeto con las bibliotecas del sistema precompiladas (una biblioteca C y `libc++`). Por eso un programa pequeño que usa `iostream`, `std::map` y `std::string` termina con cientos de funciones (592 en esta compilación).
3. **Binaryen wasm-opt:** optimiza y reduce el binario cuando usas `-O2` o superior.
4. **Generador del glue JS:** escribe `hands_on1.js` a la medida de lo que el programa importa/exporta, más el código cargador para navegador o Node.

3. Anatomía del binario real (hands_on1.wasm)

Los números siguientes se obtuvieron analizando byte por byte el `hands_on1.wasm` entregado. Muestran qué es realmente «el binario».

3.1 Encabezado y secciones

El archivo comienza con el encabezado de 8 bytes `00 61 73 6D 01 00 00 00`: el número mágico (`\0asm`) y la versión 1. Después vienen las *secciones*, cada una guardada como *id*, *tamaño*, *contenido*:

Sección	Bytes	%	Significado
type	393	0.27	Firmas de funciones que usa el módulo (p. ej. «(i32) → i32»).
import	185	0.13	7 cosas que debe proveer el anfitrión (ver 3.3).
function	594	0.41	Qué firma tiene cada una de las 592 funciones.
table	7	0.00	Tabla de llamadas indirectas (llamadas virtuales, apuntadores a función).
memory	6	0.00	Memoria lineal: 256 páginas, mín = máx (ver 3.2).
global	8	0.01	Variables globales (p. ej. el apuntador de pila de C).
export	278	0.19	Lo que el módulo ofrece a JavaScript (ver 3.4).
element	610	0.42	Contenido inicial de la tabla de llamadas indirectas.
code	131,323	90.55	Los cuerpos compilados de las 592 funciones: 90.5 % del archivo.
data	11,589	7.99	35 segmentos de datos: cadenas literales (mensajes de error, claves JSON, nombres de estado) copiadas a la memoria al arrancar.
(marco)	36	0.02	Encabezado de 8 bytes + bytes de id/tamaño de cada sección.
Total	145,029	100	

Consejo: Casi todo el peso está en `code`: es sobre todo la biblioteca estándar de C++ (iostream, string, map, set, optional, formato) que se enlazó, no tus ~10 KB de fuente. Es normal en C++ compilado a wasm y explica por qué el binario pesa 145 KB.

3.2 Memoria lineal

La sección de memoria declara **256 páginas**. Una página wasm mide 64 KiB, así que el módulo posee **16 MiB**. El mínimo y el máximo son 256, por lo tanto la memoria es **fija**: no puede crecer. Es el valor por defecto de Emscripten cuando no se usa `-sALLOW_MEMORY_GROWTH`. Nuestro programa guarda solo la línea de entrada y una cadena JSON, así que 16 MiB sobran; un programa que reserve mucho abortaría por falta de memoria salvo que se compile con crecimiento habilitado.

3.3 Importaciones (lo que debe proveer el anfitrión)

Importación	Por qué está
<code>__cxa_throw</code>	Lanzamiento de excepciones C++ en rutas de libcpp; lo implementa el glue.
<code>emscripten_memcpy_big</code> , <code>emscripten_resize_heap</code>	Copia masiva rápida y solicitud de crecer el heap (esta falla aquí porque la memoria es fija).
<code>abort</code>	Se llama cuando el runtime encuentra un error irreparable.
<code>environ_sizes_get</code> , <code>environ_get</code>	El arranque del runtime de C pide variables de entorno (el glue responde con un conjunto vacío).
<code>strftime_l</code>	Formato de fecha/localización que arrastra iostream.

3.4 Exportaciones (lo que JavaScript puede llamar)

Exportación	Tipo	Significado
h1_limpiar, h1_byte, h1_procesar, h1_leer	función	Nuestra interfaz (sección 5). Presentes gracias a <code>EMSCRIPTEN_KEEPALIVE</code> .
analizar	función	Punto de entrada antiguo basado en cadena, conservado por compatibilidad (requiere un apuntador).
memory	memoria	La memoria lineal de 16 MiB.
__wasm_call_ctors	función	Ejecuta los constructores estáticos de C++ (construye las tablas R , ACEPTA).
__indirect_function_table, __errno_location, stackSave / stackRestore / stackAlloc, __cxa_is_pointer_type, dynCall_*	varios	Infraestructura del runtime generada por Emscripten (manejo de pila, errno, llamadas indirectas). index.html no las usa directamente.

3.5 Un módulo de 41 bytes que puedes leer completo

Para quitarle misterio al formato, aquí hay un módulo ensamblado a mano que exporta `add(a, b)`. Se ejecutó y devuelve `add(2,3) = 5`.

```
00 61 73 6D 01 00 00 00      encabezado: mágico "\0asm" + versión 1
01 07 01 60 02 7F 7F 01 7F  sección type: 1 tipo = func(i32,i32) -> i32
03 02 01 00                  sección function: 1 función, usa el tipo 0
07 07 01 03 61 64 64 00 00  sección export: "add" (bytes 61 64 64) = función 0
0A 09 01 07 00 20 00 20 01 6A 0B  sección code: local.get 0, local.get 1, i32.add, end
```

```
const bytes = new Uint8Array([0x00,0x61,0x73,0x6d,1,0,0,0, 1,7,1,0x60,2,0x7f,0x7f,1,0x7f, 3,2,1,0,
7,7,1,3,0x61,0x64,0x64,0,0, 0x0a,9,1,7,0,0x20,0,0x20,1,0x6a,0x0b]);
const { instance } = await WebAssembly.instantiate(bytes);
console.log(instance.exports.add(2, 3)); // 5
```

Las instrucciones `20 00`, `20 01`, `6A` son una *máquina de pila*: empuja el parámetro 0, empuja el parámetro 1, suma los dos valores de arriba. Tu `hands_on1.wasm` es el mismo formato, solo que con 592 funciones en lugar de una.

3.6 hands_on1.wasm frente a main.wasm

Aspecto	hands_on1.wasm	main.wasm
Tamaño	145,029 B	265,885 B
Nuestras exportaciones	h1_limpiar, h1_byte, h1_procesar, h1_leer, analizar	solo analizar
Importaciones extra	ninguna de salida	fd_write, fd_close, fd_seek (consola/stdio)
Referenciado por index.html / hands_on1.js	Sí (el nombre está escrito dentro del .js)	No

Conclusión: `main.wasm` es otra compilación y la página nunca la carga. Si se renombrara a `hands_on1.wasm`, la página mostraría el `.wasm` no trae `h1_procesar` porque esa exportación no existe en él.

4. Preparar el compilador (Emscripten SDK)

4.1 Requisitos

- Git y Python 3 disponibles en la terminal (el instalador del SDK es un script de Python).
- Acceso a internet durante la instalación (descarga clang, Binaryen, Node.js y las fuentes de Emscripten). Se aconsejan varios GB de disco.

4.2 Instalación (emsdk)

```
git clone https://github.com/emscripten-core/emsdk.git
cd emsdk
./emsdk install latest           # Windows (cmd/PowerShell): emsdk install latest
./emsdk activate latest         # emsdk activate latest
source ./emsdk_env.sh           # Windows cmd: emsdk_env.bat PowerShell: .\emsdk_env.ps1
emcc --version                  # imprime la versión de Emscripten si todo está en el PATH
```

Nota: El último paso `emsdk_env` solo afecta a la terminal **actual**. Si abres una terminal nueva y aparece `emcc: command not found`, ejecútalo otra vez (o agrega las rutas de forma permanente).

4.3 Prueba rápida antes de tocar el proyecto

```
// hello.cpp
#include <iostream>
int main() { std::cout << "hola wasm\n"; }

emcc hello.cpp -o hello.js
node hello.js           # imprime: hola wasm (Node ejecuta el .js, que carga hello.wasm)
```

4.4 g++ frente a emcc

Aspecto	g++	emcc
Destino	Tu CPU + SO	wasm32 (portable)
Salida	Ejecutable (.exe / ELF)	.wasm + .js (o .html)
Quién lo ejecuta	El sistema operativo	Navegador o Node.js
Punto de entrada	<code>main()</code>	Funciones exportadas (este proyecto) o <code>main()</code>
Uso típico aquí	Pruebas rápidas por consola	La versión web

5. Preparar el código C++ para el navegador

5.1 Un código, dos compilaciones: `#ifdef __EMSCRIPTEN__`

Emscripten define automáticamente la macro `__EMSCRIPTEN__`. La práctica la usa para elegir el punto de entrada en tiempo de compilación:

```
#ifdef __EMSCRIPTEN__
// ... funciones exportadas para JavaScript (sección 5.2) ...
#else
int main(int argc, char** argv) { // compilación g++: programa de línea de comandos
    bool js = false; string in;
    for (int i = 1; i < argc; ++i) { string a = argv[i]; if (a == "--json") js = true; else in += ... }
    if (in.empty()) getline(cin, in);
    Res r = procesar(in);
    if (js) cout << aJson(r) << "\n"; else imprimir(r);
}
#endif
```

Ambas compilaciones comparten exactamente el mismo motor (`procesar()`, `aJson()`), así que lo que pruebas en consola es lo que corre en el navegador.

5.2 La interfaz exportada

```
static string g_in, g_out; // estado que vive mientras exista la instancia wasm
extern "C" {
    EMSCRIPTEN_KEEPALIVE void h1_limpiar() { g_in.clear(); }
    EMSCRIPTEN_KEEPALIVE void h1_byte(int b) { g_in.push_back((char)b); }
    EMSCRIPTEN_KEEPALIVE int h1_procesar() { g_out = aJson(procesar(g_in)); return (int)g_out.size(); }
    EMSCRIPTEN_KEEPALIVE int h1_leer(int i) { return (unsigned char)g_out[i]; }
    EMSCRIPTEN_KEEPALIVE const char* analizar(const char* s) { // versión anterior, basada en cadenas
        static string out; out = aJson(procesar(s)); return out.c_str(); }
}
```

Pieza	Para qué sirve
<code>extern "C"</code>	C++ normalmente <i>decora</i> los nombres de función (p. ej. <code>_Z10h1_procesarv</code>) para soportar sobrecarga. <code>extern "C"</code> conserva el nombre simple <code>h1_procesar</code> para que JavaScript lo encuentre.
<code>EMSCRIPTEN_KEEPALIVE</code>	Indica al compilador/enlazador que no elimine la función por «no usada» y que la exporte. Sin esto solo sobreviviría <code>main</code> . Viene de <i>emscripten.h</i> .
<code>g_in / g_out</code>	Globales estáticas que persisten entre llamadas, así JavaScript puede llenar la entrada en varias llamadas y leer la salida después.
<code>h1_procesar()</code>	Ejecuta todo el flujo y devuelve la longitud del JSON para que JS sepa cuántos bytes leer.

5.3 Por qué solo enteros cruzan la frontera

Una cadena de JavaScript no se puede entregar directamente a C++: las funciones *wasm* solo reciben números. Para pasar texto normalmente se necesitan **auxiliares de reserva de memoria** (`malloc`, `stringToUTF8`) y **vistas del heap** (`HEAPU8`) del runtime de Emscripten, y esos solo existen si los pides con banderas de enlace (`-sEXPORTED_RUNTIME_METHODS=...`, `-sEXPORTED_FUNCTIONS=...`). La práctica evita esa dependencia: JavaScript envía el texto **un byte a la vez** (`h1_byte`) y lee el JSON **un byte a la vez** (`h1_leer`). Solo cruzan valores `i32`, así que la interfaz funciona con *cualquier* bandera de `emcc`. El comentario del código lo dice: no depende de `cwrap`, `ccall`, `malloc` ni `HEAPU8`.

5.4 La alternativa: `cwrap`

Compilar con `-sEXPORTED_RUNTIME_METHODS=cwrap` agrega el auxiliar `Module.cwrap`, que envuelve una función C (incluida la conversión de cadenas) en una función JS normal:

```
const analizar = Module.cwrap('analizar', 'string', ['string']);  
const json = analizar("MOV AL, 6");      // JS más simple, pero depende de la bandera de enlace
```

Nota: En el `hands_on1.js` entregado la palabra `cwrap` no aparece, así que esa compilación se hizo **sin** esa bandera. El comando del README todavía la menciona. Es inofensivo para la página (usa las funciones enteras) pero conviene que README y comando de compilación coincidan.

6. Compilar paso a paso

6.1 Primero, prueba de forma nativa con g++

Valida siempre la lógica antes de involucrar al navegador: los errores se leen mucho mejor en una terminal.

```
g++ -std=c++17 -O2 HandsOn1_gonzalez_martinez.cpp -o hands_on1
./hands_on1 "MOV AL, 6"          # reporte de texto
./hands_on1 --json "STO 8"       # el mismo JSON que recibe la página
```

Resultado medido para `MOV AL, 6`: tokens `MOV`, `REGISTRO("AL")`, `COMA`, `NUMERO("6")`; objeto `{operacion:"MOV", registros:["AL"], direccion:6}`; microoperaciones `MAR ← 6`; `MBR ← M[MAR]`; `AL ← MBR`. Para `MOV AX, 6`: inválida, «registro inválido: MOV solo admite AL o BL».

6.2 Compilar a WebAssembly

```
emcc HandsOn1_gonzalez_martinez.cpp -std=c++17 -O2 -o hands_on1.js
```

Por `-o hands_on1.js`, emcc escribe **dos** archivos en la carpeta actual: `hands_on1.js` y `hands_on1.wasm` (mismo nombre base). Compruébalos:

```
ls -l hands_on1.js hands_on1.wasm      # Windows: dir hands_on1.*
# esperado en este proyecto: hands_on1.js ~22.7 KB, hands_on1.wasm ~145 KB
```

6.3 Las banderas, una por una

Bandera	Qué hace	¿Se usó en la práctica?
-std=c++17	Selecciona el estándar del lenguaje. Necesaria porque el código usa <code>std::optional</code> .	Sí
-O2	Nivel de optimización: velocidad y tamaño. También activa <code>wasm-opt</code> de Binaryen. (-O0 ninguna, -O1, -O2, -O3 más agresiva, -Os / -Oz más pequeño.)	Sí
-o hands_on1.js	Nombre de salida; su extensión decide el tipo: <code>.js</code> → <code>js+wasm</code> , <code>.html</code> → <code>js+wasm+cascarón HTML</code> , <code>.wasm</code> → solo <code>wasm</code> (independiente).	Sí
-sEXPORTED_RUNTIME_METHODS=cwrap	Expone el auxiliar JS <code>Module.cwrap</code> .	Solo en el README; no está en la compilación entregada
-sEXPORTED_FUNCTIONS=_name,...	Exporta funciones sin necesitar <code>EMSCRIPTEN_KEEPALIVE</code> .	No (se usó <code>KEEPALIVE</code>)
-sALLOW_MEMORY_GROWTH=1	Permite que la memoria crezca más allá de los 16 MiB iniciales.	No (la memoria es fija)
-sMODULARIZE=1 -sEXPORT_NAME=X	Envuelve el glue en una función fábrica en lugar de un <code>Module</code> global.	No
-sSINGLE_FILE=1	Incrusta el <code>wasm</code> dentro del <code>.js</code> en base64 (un archivo, ~33 % más grande).	No
-g / -sASSERTIONS=1	Información de depuración y comprobaciones extra durante el desarrollo.	No

6.4 Los nombres deben coincidir

- El glue contiene el literal `wasmBinaryFile="hands_on1.wasm"`. Si renombras el `.wasm`, la página se rompe; recompila con otro `-o` en lugar de renombrar.

- El archivo fuente del zip es `HandsON1_gonzalez_martinez.cpp` pero los comentarios y el README dicen `hands_on1.cpp`. Usa el nombre real en el comando, o renombra el archivo de forma consistente.
- Siempre que cambies el C++ debes recompilar **y** volver a subir `hands_on1.js` y `hands_on1.wasm` (son un par que va junto).

7. Cómo el glue carga y arranca el binario

Esto es lo que hace el `hands_on1.js` generado, siguiendo el código real del archivo entregado.

1. **Toma o crea Module:** `var Module = typeof Module != "undefined" ? Module : {}`. Si la página definió `Module` antes de cargar el script, esa configuración (como `onRuntimeInitialized`) se conserva. Por eso `index.html` declara `var Module = {...}` primero.
2. **Localiza el binario:** `wasmBinaryFile = "hands_on1.wasm"` resuelto relativo a la URL de la página (`locateFile`).
3. **Arma las importaciones:** `info = {env: wasmImports, wasi_snapshot_preview1: wasmImports}` — las 7 funciones de la sección 3.3.
4. **Descarga y compila:** si existe `WebAssembly.instantiateStreaming` y la página *no* se abrió desde `file://`, lo hace en flujo (compila mientras descarga). Si no, o si el flujo falla, hace `fetch(...).arrayBuffer()` seguido de `WebAssembly.instantiate`. Ante un fallo de flujo escribe `falling back to ArrayBuffer instantiation`.
5. **Recibe la instancia:** `Module.asm = instance.exports`; toma `memory` y construye las vistas del heap.
6. **Ejecuta los constructores:** `__wasm_call_ctors` construye las tablas estáticas `R` y `ACEPTA` y los demás objetos estáticos.
7. **Avisa que está listo:** llama a `Module.onRuntimeInitialized()` — en este proyecto, `boot()`.

7.1 Envoltorios de exportación

Para cada exportación el glue define una función delgada. Se enlazan de forma perezosa a la exportación real la primera vez que se llaman:

```
var _h1_procesar = Module["_h1_procesar"] = function() {
  return (_h1_procesar = Module["_h1_procesar"] =
    Module["asm"]["h1_procesar"]).apply(null, arguments);
};
// En JS lleva guion bajo (_h1_procesar); dentro del wasm es simple (h1_procesar)
```

7.2 Flujo frente a ArrayBuffer

Camino	Requisitos	Comportamiento
<code>instantiateStreaming</code>	Origen HTTP(S); cabecera <code>Content-Type: application/wasm</code>	Compila mientras descarga: arranque más rápido.
<code>fetch + instantiate</code>	Origen HTTP(S); cualquier tipo de contenido	Descarga todo y luego compila. Camino de respaldo.
<code>file://</code>	—	No funciona: los navegadores bloquean leer archivos locales desde una página (el propio glue omite <code>fetch</code> para URIs de archivo).

8. index.html: conectar la interfaz con el motor

8.1 El orden de los scripts importa

```
<script>
/* ... código de la interfaz ... */
function boot(){ ... } // se llama cuando el wasm está listo
var Module = { onRuntimeInitialized: boot,
                onAbort: w => { $('st').textContent = '· WASM abortó: ' + w } };
</script>
<script src="hands_on1.js" // ruta relativa, misma carpeta
        onerror="$('st').textContent='· no se pudo cargar hands_on1.js ...'"></script>
```

`Module` se define **antes** de cargar el glue para que éste recoja los callbacks. La ruta relativa `src="hands_on1.js"` funciona tanto en `localhost` como en <https://usuario.github.io/repo/>.

8.2 boot() y la función puente an()

```
function boot(){
  const M = Module;
  if (typeof M._hl_procesar !== 'function') throw new Error('el .wasm no trae hl_procesar: ...');
  const enc = new TextEncoder(), dec = new TextDecoder();
  window.an = s => {
    M._hl_limpiar(); // 1. limpiar el búfer de entrada de C++
    for (const b of enc.encode(s)) M._hl_byte(b); // 2. texto -> bytes UTF-8 -> wasm
    const n = M._hl_procesar(); // 3. corre C++; devuelve long. del JSON
    const u = new Uint8Array(n);
    for (let i = 0; i < n; i++) u[i] = M._hl_leer(i); // 4. leer el JSON byte a byte
    return dec.decode(u); // 5. bytes -> cadena JS
  };
  $('st').textContent = '· listo'; run();
}
```

Se usa UTF-8 en ambos lados, por eso caracteres como ← y las letras con acento viajan correctamente. Luego `run()` hace `D = JSON.parse(an(entrada))` y el resto de la página solo dibuja `D`.

8.3 El contrato JSON

Campo	Contenido	Se usa para
entrada, ok, error	Entrada limpia, bandera de validez, mensaje de error.	Encabezado y panel de error
pasos[]	Cada paso del AFD: carácter <code>c</code> , estado origen <code>de</code> , estado destino <code>a</code> .	Reproductor de pasos, resaltado del grafo
tokens[]	<code>tipo</code> , <code>lex</code> , <code>paso</code>	Lista de tokens que aparece progresivamente
objeto	<code>operacion</code> , <code>registros</code> , <code>direccion</code> (o null si es inválida)	Panel del objeto instrucción
moore[]	<code>estado</code> , <code>salida</code> , <code>sig</code>	Lista de Moore y animación del camino de datos
afd	<code>aristas[]</code> , <code>acepta[]</code>	Dibuja el grafo desde la tabla de transiciones real

Medido: el JSON de `MOV AL, 6` mide 1,717 bytes, porque incluye la lista completa de aristas del AFD en cada llamada. Por eso el grafo de la página se construye siempre con la misma tabla que usa el C++: nunca puede discrepar del motor.

Nota: Los valores numéricos del panel de registros ($AL=5$, $BL=7$, $M[d]=(7d+3) \bmod 100$) los calcula la función JavaScript `machine()`, solo para ilustrar. El `wasm` produce el *texto* de cada microoperación, no la aritmética.

9. Probar localmente antes de publicar

9.1 Servir la carpeta por HTTP

```
cd carpeta-con-los-tres-archivos
python3 -m http.server 8000          # Windows: py -m http.server 8000
# abrir http://localhost:8000/index.html
```

No hagas doble clic en `index.html`: una página `file://` no puede leer `hands_on1.wasm` (sección 7.2). Medido con el servidor integrado de Python 3.12: `hands_on1.wasm` se entrega con `Content-Type: application/wasm` y `Content-Length: 145029`, así que la compilación en flujo funciona.

9.2 Verificaciones en el navegador (DevTools, F12)

- **Network (Red):** tres peticiones con estado 200: `index.html`, `hands_on1.js`, `hands_on1.wasm` (tipo `wasm`, ~145 kB).
- **Console (Consola):** sin errores en rojo. Escribe `typeof Module._h1_procesar` → debe responder `"function"`.
- **Página:** el subtítulo cambia de «(cargando...)» a «· listo».

9.3 Prueba sin navegador con Node.js

El mismo glue corre en Node, lo que permite una prueba automática del binario real:

```
// test.js -> node test.js
const M = require('./hands_on1.js');
M.onRuntimeInitialized = () => {
  const enc = new TextEncoder(), dec = new TextDecoder();
  const an = s => { M._h1_limpiar(); for (const b of enc.encode(s)) M._h1_byte(b);
    const n = M._h1_procesar(), u = new Uint8Array(n);
    for (let i = 0; i < n; i++) u[i] = M._h1_leer(i); return dec.decode(u); };
  const j = JSON.parse(an("MOV AL, 6"));
  console.log(j.ok, j.tokens.map(t => t.tipo + ':' + t.lex).join(' '),
    j.moore.map(m => m.salida).join(' ; '));
  console.log(JSON.parse(an("MOV AX, 6")).error);
};
```

Salida obtenida con los archivos entregados:

```
true MOV:MOV REGISTRO:AL COMA:, NUMERO:6 MAR ← 6 ; MBR ← M[MAR] ; AL ← MBR ;
registro inválido: MOV solo admite AL o BL
```

9.4 Matriz de pruebas

Entrada	Resultado esperado
MOV AL, 6	válida · MAR ← 6; MBR ← M[MAR]; AL ← MBR
MOV BL, 7	válida · MAR ← 7; MBR ← M[MAR]; BL ← MBR
ADD AL, BL	válida · ACC ← AL + BL
STO 8	válida · MAR ← 8; MBR ← ACC; M[MAR] ← MBR
END	válida · HALT ← 1
MOV AX, 6	inválida · registro inválido: MOV solo admite AL o BL
MOV AL 6	inválida · falta la coma entre los operandos
ADD AL,	inválida · falta el registro BL

Entrada	Resultado esperado
ST0	inválida · falta la dirección de memoria en ST0
END 8	inválida · END no admite operandos

Toma una captura de cada caso en el navegador para la columna «Evidencia» que el README marca como pendiente.

10. Git y GitHub: publicar los archivos

10.1 Qué subir

Grupo	Archivos	Motivo
Obligatorios	index.html, hands_on1.js, hands_on1.wasm	El sitio en sí. Deben estar en la misma carpeta (rutas relativas).
Recomendados	HandsON1_gonzalez_martinez.cpp, README.md	Fuente y documentación: el repositorio debe permitir a cualquiera reconstruir el wasm.
No necesarios	main.wasm, hands_on1.zip	main.wasm no se usa; el zip es solo un formato de entrega, GitHub no lo necesita.

10.2 Configuración inicial (una vez)

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu@ejemplo.com"
# En github.com: New repository -> ponle nombre (p. ej. hands-on1) -> Public -> Create
```

Nota: GitHub Pages está disponible para repositorios **públicos** en el plan Gratis; los repositorios privados requieren un plan de pago (documentación «Pages limits» de GitHub).

10.3 Commit y push

```
cd carpeta-con-los-archivos
git init
git add index.html hands_on1.js hands_on1.wasm HandsON1_gonzalez_martinez.cpp README.md
git commit -m "Hands-on 1: motor C++ compilado a WebAssembly"
git branch -M main
git remote add origin https://github.com/<usuario>/<repo>.git
git push -u origin main
```

Comando	Qué ocurre
git init	Crea la carpeta oculta <code>.git</code> : la base de datos del historial local.
git add	Marca archivos para incluirlos en la siguiente instantánea (el «área de preparación»).
git commit	Guarda la instantánea, con un mensaje, en el historial local.
git branch -M main	Nombra la rama <code>main</code> (la que publicará Pages).
git remote add origin	Registra la URL de GitHub con el apodo «origin».
git push -u origin main	Sube los commits y enlaza el <code>main</code> local con <code>origin/main</code> .

Autenticación: al hacer push por HTTPS, GitHub pide un **token de acceso personal** (o el Git Credential Manager te autentica mediante el navegador); las contraseñas de cuenta ya no se aceptan para operaciones Git. Las llaves SSH son la alternativa. Nunca escribas un token dentro de un archivo que vayas a subir.

10.4 Proteger el binario y los finales de línea

Git detecta archivos binarios buscando bytes cero (un wasm empieza con `00 61 73 6D`), pero puedes hacerlo explícito y evitar cualquier conversión de saltos de línea agregando un archivo `.gitattributes`:

```
*.wasm binary
```

```
*.js  text eol=lf
```

Los archivos de texto entregados usan finales de línea de Windows (CRLF); es inofensivo para el navegador, solo cosmético en los diffs.

10.5 Actualizar el sitio después

```
# editar el .cpp -> recompilar -> verificar localmente -> publicar
emcc HandsON1_gonzalez_martinez.cpp -std=c++17 -O2 -o hands_on1.js
git add hands_on1.js hands_on1.wasm HandsON1_gonzalez_martinez.cpp
git commit -m "Describe el cambio"
git push
```

Si prefieres la interfaz web: *Add file* → *Upload files* en la página del repositorio sirve para archivos pequeños como éstos.

11. GitHub Pages: cómo se sirve el binario

11.1 Activarlo

1. Abre el repositorio en github.com → **Settings** → **Pages**.
2. En *Build and deployment*, *Source*: **Deploy from a branch**.
3. Branch: **main**, carpeta: / (**root**) → **Save**.
4. Espera el despliegue (la pestaña Actions muestra «pages build and deployment»). El sitio aparece en <https://<usuario>.github.io/<repo>/>.
5. Abre <.../<repo>/index.html> (o solo <.../<repo>/>, pues index.html es el documento por defecto).

11.2 Qué es (y qué no es) Pages

GitHub Pages es **alojamiento estático**: cada vez que haces push a la rama de origen se publican los archivos de la carpeta de origen. No ejecuta tu código en el servidor. Al publicar desde una rama usa Jekyll por defecto para construir el sitio; con archivos simples como los nuestros Jekyll solo los copia. Puedes agregar un archivo vacío llamado `.nojekyll` para omitir ese paso. Como alternativa, un flujo de trabajo propio de GitHub Actions puede construir y publicar.

11.3 Resolución de URL: la trampa de los sitios de proyecto

Un sitio de proyecto vive bajo el nombre del repositorio: <https://usuario.github.io/hands-on1/>. La referencia relativa `src="hands_on1.js"` se resuelve a `/hands-on1/hands_on1.js` ✓. Una referencia absoluta `src="/hands_on1.js"` apuntaría a la raíz del dominio `/hands_on1.js` ✗ (404). El glue también resuelve `hands_on1.wasm` relativo a la página, así que mantén todo relativo. Recuerda que los servidores de GitHub **distinguen mayúsculas**: `Hands_on1.wasm` es un archivo distinto de `hands_on1.wasm`.

11.4 Tipo de contenido, compresión y caché

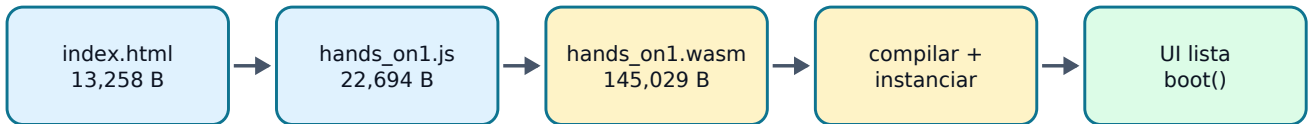
- **Content-Type**: Pages lo decide por la extensión del archivo; no puedes definir tipos MIME personalizados por archivo ni por repositorio. Un `application/wasm` correcto habilita la compilación en flujo. Verifícalo en DevTools → Network → `hands_on1.wasm` → Headers. Si el tipo fuera incorrecto el glue recurriría al camino `ArrayBuffer` y seguiría funcionando.
- **Compresión**: medido con `gzip -9` sobre los archivos entregados: `hands_on1.wasm` 145,029 → 55,171 bytes (38 %), `hands_on1.js` 22,694 → 6,820 bytes (30 %). Si una respuesta concreta viene comprimida se ve en la cabecera *Content-Encoding* y en la columna *Transferred* frente a *Size* de Network. No necesitas comprimir los archivos tú mismo.
- **Caché**: navegadores y CDN pueden conservar copias viejas. Tras publicar una compilación nueva usa recarga forzada (Ctrl+F5, o Cmd+Shift+R). La documentación de GitHub sugiere revisar errores de compilación si los cambios no aparecen después de una hora.
- **HTTPS**: Pages sirve por HTTPS, lo que además garantiza que la página sea un contexto seguro.

11.5 Límites (documentación de GitHub)

Límite	Valor	Relevancia aquí
Tamaño del sitio publicado	no mayor a 1 GB	El nuestro: ~0.2 MB
Repositorio de origen	recomendado 1 GB	Sin problema
Ancho de banda	límite suave 100 GB / mes	~175 KB por primera visita
Compilaciones	límite suave 10 por hora	Evita hacer muchos push seguidos
Tiempo máximo de despliegue	10 minutos	Sin problema

Nota: Los sitios de Pages publicados son públicos. No incluyas credenciales, datos personales ni nada privado en el repositorio ni en la página.

12. Qué ocurre cuando alguien abre la URL



#	Momento	Qué sucede
1	Conexión	DNS resuelve <i>usuario.github.io</i> ; se abre una sesión TLS (HTTPS).
2	GET index.html	El navegador descarga y analiza el HTML y el CSS; corre el primer script en línea y define <code>Module</code> .
3	GET hands_on1.js	El segundo <code><script src></code> descarga y ejecuta el glue. El glue empieza a pedir el binario.
4	GET hands_on1.wasm	El navegador recibe los bytes; comprueba el número mágico, valida cada función y sección, y compila a código máquina (en flujo si es posible).
5	Instanciación	Se crea la memoria de 16 MiB, se copian los 35 segmentos de datos (cadenas), las 7 importaciones se enlazan con funciones JS, las exportaciones quedan disponibles.
6	Arranque	<code>__wasm_call_ctors</code> construye las tablas estáticas; el glue llama a <code>onRuntimeInitialized</code> → <code>boot()</code> .
7	Primer análisis	<code>boot()</code> define <code>an()</code> y llama a <code>run()</code> con el texto por defecto <code>MOV AL, 6</code> .
8	Cada clic en Analizar	JS → bytes → <code>h1_*</code> → C++ → JSON → <code>JSON.parse</code> → SVG/DOM. Todo ocurre en el navegador del visitante; GitHub ya no interviene.

12.1 Por qué es seguro ejecutarlo

- El módulo se **valida** antes de ejecutarse; el código mal formado o inseguro en tipos se rechaza.
- Solo toca su propia memoria lineal; los accesos fuera de rango provocan una trampa en lugar de leer otra memoria.
- Su única vía al exterior son las 7 importaciones que da el glue (ninguna da acceso a archivos ni red).
- Corre bajo las reglas de mismo origen del navegador, como cualquier script de esa página.

13. Cómo se construyó la práctica (procedimiento)

Éste es el orden en que está organizada la práctica, reconstruido a partir de los archivos entregados y el README. Se puede seguir como receta.

#	Paso	Dónde / resultado
1	Definir el ISA de referencia (MOV, ADD, STO, END), registros y microoperaciones.	README §2
2	Diseñar el AFD como tabla explícita de transiciones (estado, carácter, siguiente estado) y los estados de aceptación.	R, ACEPTA
3	Escribir el motor: sig , tipo , msg , procesar , Instr , clase Moore .	.cpp
4	Agregar las dos salidas: reporte de texto (imprimir) y JSON (aJson).	.cpp
5	Probar en consola con g++ con entradas válidas e inválidas.	sección 6.1, 9.4
6	Agregar la interfaz de Emscripten dentro de #ifdef __EMSCRIPTEN__ (funciones solo con enteros).	sección 5
7	Compilar con emcc → hands_on1.js + hands_on1.wasm .	sección 6.2
8	Escribir index.html : grafo, tokens, objeto, Moore, camino de datos, reproductor, modo desafío.	index.html
9	Servir localmente y verificar en el navegador.	sección 9
10	Subir los tres archivos (más fuente y README) a GitHub y activar Pages.	secciones 10–11
11	Documentar: README con casos de prueba y capturas de evidencia.	README §9

Consejo: El archivo contiene dos compilaciones: **main.wasm** (solo analizar) y **hands_on1.wasm** (con **h1_***). Junto con el comentario del código sobre «no depender de cwrap», es consistente con que la interfaz se cambió de una llamada con cadenas a la de solo enteros. Solo **hands_on1.wasm** forma parte del sitio final.

14. Solución de problemas

Síntoma	Causa probable	Solución
La página queda en «(cargando...)» o «no se pudo cargar hands_on1.js»	Nombre o mayúsculas incorrectos, archivo no subido o carpeta equivocada (Network muestra 404).	Haz coincidir nombres y mayúsculas exactamente; mantén los tres archivos juntos.
Funciona en servidor pero no con doble clic	file:// bloquea la carga del .wasm.	Usa <code>python3 -m http.server</code> o GitHub Pages.
<code>expected magic word 3c 21 44 4f</code>	El servidor devolvió una página HTML (404) en lugar del binario: los bytes 3c 21 44 4f son «<!DO».	Corrige la ruta/nombre del .wasm.
<code>el .wasm no trae h1_procesar</code>	Wasm viejo o distinto (p. ej. main.wasm renombrado) o compilado con un fuente antiguo.	Recompila el .cpp actual y sube el .js y el .wasm.
<code>falling back to ArrayBuffer instantiation</code>	Falló la compilación en flujo (normalmente Content-Type incorrecto).	Aun así funciona; revisa la cabecera si quieres flujo.
Sigue la versión vieja tras el push	Caché, o el despliegue no terminó.	Espera la ejecución en Actions; recarga forzada Ctrl+F5.
<code>emcc: command not found</code>	El entorno de emsdk no está cargado en esta terminal.	Ejecuta <code>emsdk_env</code> otra vez.
<code>Module.cwrap is not a function</code>	Compilado sin <code>-sEXPORTED_RUNTIME_METHODS=cwrap</code> .	Usa las funciones enteras <code>_h1_*</code> o agrega la bandera.
Caracteres extraños (Ã©, â†)	Falta la declaración UTF-8.	Conserva <code><meta charset="utf-8"></code> y el par <code>TextEncoder/TextDecoder</code> .
Aborta por falta de memoria con entradas enormes	La memoria es fija de 16 MiB.	Compila con <code>-sALLOW_MEMORY_GROWTH=1</code> .
Pages da 404 en todo el sitio	Pages sin activar, rama/carpeta equivocada o repositorio privado en plan Gratis.	Revisa Settings → Pages (sección 11.1).

Apéndice A. Resumen de comandos

```
# --- Preparar el compilador (una vez) ---
git clone https://github.com/emscripten-core/emsdk.git && cd emsdk
./emsdk install latest && ./emsdk activate latest && source ./emsdk_env.sh
emcc --version

# --- Prueba nativa ---
g++ -std=c++17 -O2 HandsOn1_gonzalez_martinez.cpp -o hands_on1
./hands_on1 "MOV AL, 6" ./hands_on1 --json "STO 8"

# --- Compilación a WebAssembly ---
emcc HandsOn1_gonzalez_martinez.cpp -std=c++17 -O2 -o hands_on1.js

# --- Servidor local ---
python3 -m http.server 8000 # http://localhost:8000/index.html

# --- Publicar ---
git init && git add . && git commit -m "Hands-on 1" && git branch -M main
git remote add origin https://github.com/<usuario>/<repo>.git && git push -u origin main
# GitHub: Settings -> Pages -> Deploy from a branch -> main / (root)
```

Lista de verificación antes de entregar

- Los tres archivos están en la misma carpeta y los nombres coinciden exactamente (mayúsculas incluidas).
- `hands_on1.js` y `hands_on1.wasm` provienen de la **misma** compilación.
- Prueba local con servidor: 5 entradas válidas + 5 inválidas se comportan como en la sección 9.4.
- Los comandos del README usan el nombre real del archivo fuente y coinciden con las banderas realmente usadas.
- Capturas de evidencia agregadas a las tablas de pruebas del README.
- La URL de Pages abre en una ventana privada (sin caché) y muestra «· listo».

Apéndice B. Glosario

Término	Significado
WebAssembly (wasm)	Formato binario de instrucciones portable que ejecutan los navegadores en un entorno aislado.
Emscripten / emcc	Cadena de herramientas y controlador que convierte C/C++ en wasm + glue JS.
LLVM / clang	Infraestructura de compilación y front-end de C/C++ que usa Emscripten.
Binaryen / wasm-opt	Optimizador de módulos wasm.
Glue (hands_on1.js)	JS generado que carga y conecta el wasm con la página.
Linear memory	El arreglo de bytes privado del módulo; los apuntadores son desplazamientos en él.
Import / Export	Función que el anfitrión da al módulo / función que el módulo da al anfitrión.
EMSCRIPTEN_KEEPALIVE	Macro que conserva y exporta una función.
MIME type	Etiqueta (Content-Type) que dice al navegador qué es un archivo descargado; wasm → application/wasm.
GitHub Pages	Servicio de alojamiento estático que publica una rama/carpeta de un repositorio por HTTPS.
Static constructors	Objetos C++ contruidos antes de usarse; wasm los ejecuta con <code>__wasm_call_ctors</code> .

